VEKTOR

For technical due diligence teams, CTOs, and institutional partners

Vektor Research Series · IP-WP-ARCH-260501-1.0 ·

May 2026

ABSTRACT

Most vendor technical documentation is a list of services used. This paper is a record of decisions made — including the reasoning behind each one and what it would take to change them. This paper documents the Vektor production infrastructure through that lens: the architectural decisions made at the design stage, the reasoning behind each, and the properties those decisions produce—modularity, scalability, and the ability to integrate with existing institutional infrastructure. The platform is currently deployed in Singapore (ap-southeast-1) for the initial SGX implementation. The architecture is region-agnostic by design: any AWS region can be configured as the deployment target without architectural change. Where capabilities are on the roadmap rather than live today, this paper says so directly.

KEY TAKEAWAYS

— Vektor runs on AWS using a microservices architecture. The current deployment is in ap-southeast-1 (Singapore). The architecture is region-agnostic — any AWS region can be configured as the deployment target. A client with UK or Hong Kong data residency requirements deploys to a different region; AWS data residency guarantees follow.

— SageMaker serves two distinct roles: JupyterLab research environment for universe screening, and a managed XGBoost inference endpoint for signal validation. These are separate concerns, treated separately.

— SQS decouples order generation from execution. A connectivity issue with IBKR does not cascade into the application layer.

— EventBridge + Step Functions provide observable, retryable workflow orchestration — producing an implicit audit trail for every automated operation.

— Cognito with Azure EntraID integration enables SSO for institutional firms running Microsoft 365 — designed in from the first prototype.

— The two-account AWS strategy (shared CI/CD + workload) means deploying to a new region is a CI/CD pipeline configuration, not an architecture change. CodePipeline in the shared account builds Docker images, pushes to ECR, and deploys cross-account to the workload account — making the region-agnostic claim concrete rather than aspirational.

— All infrastructure is defined in AWS CloudFormation. Every resource is version-controlled, reproducible, and auditable—no manual console configuration exists in the production path.

— ECS services run on Fargate Spot capacity, delivering approximately 70% reduction in compute costs versus on-demand pricing. Compute is one component of total infrastructure cost; the saving is meaningful at scale and reflects deliberate infrastructure cost management.

1.

PLATFORM OVERVIEW

What Vektor is built on, and why the infrastructure choices reflect the same design principles as the platform itself.

Vektor is deployed on Amazon Web Services using a microservices architecture. The platform is currently running in ap-southeast-1 (Singapore), chosen as the deployment base for the initial SGX implementation and for alignment with the MAS regulatory context of the founding client base.

> **Region-agnostic by design.** Singapore (ap-southeast-1) is the current deployment region—a configuration choice, not a structural dependency. A client with UK data residency requirements deploys to eu-west-2; Hong Kong to ap-east-1; US to us-east-1 or us-west-2. AWS data residency guarantees follow the region. The two-account CI/CD architecture means deploying to a new region is a pipeline configuration, not an architecture change.

Three design principles were established before the first line of code was written. **Modularity:** every capability is an independently deployable service. **Scalability by default:** the infrastructure scales horizontally without architectural changes. **Governance before growth:** authentication, encryption, deployment pipeline separation, and operational visibility were designed in from the start.

> **Current deployment status.** The architecture described in this document is designed for institutional production. The current deployment is at prototype stage, running in ap-southeast-1 with IBKR connected to a simulation account. Section 9 describes which specific capabilities are live today and which are on the near-term roadmap. The infrastructure designed today supports all described capabilities — implementing the roadmap items is configuration and integration work, not architectural change.

2.

ARCHITECTURE DIAGRAM

The complete Vektor AWS stack—nine layers, colour-coded by user role. Read in conjunction with the decision log in Section 5.

USERS	Stock Analyst · Portfolio Manager · Operations
FRONTEND	React Admin Portal (all three roles) · AWS Amplify hosting + CI/CD
API	AWS API Gateway · Single entry point · Auth enforced · Rate limiting on roadmap · All ECS services exposed
COMPUTE	ECS Microservices (Python / Flask) · ECS Service Connect: sub-100ms inter-service calls Fargate Spot: ~70% compute cost reduction · Portfolio svc · Allocation svc · Data svc SageMaker: JupyterLab Research + XGBoost Endpoint
MESSAGING	SQS: Market order queue · EventBridge: Scheduled triggers Step Functions: Workflow orchestration · SNS + Lambda router: Operational notifications
DATA	RDS PostgreSQL · Schema separation: market_data / portfolio_data / customer_data Equity prices + FX rates · S3: Model artefacts + Backup
EXECUTION	EC2 t3.large: IBKR Gateway + SQS consumer · IBKR API (Simulation → Live) EC2 t3.medium: Bastion host
AUTH & SECURITY	Cognito + Azure EntraID SSO · KMS: Encryption at rest · CloudWatch: Logging
INFRASTRUCTURE	AWS ap-southeast-1 (Singapore) · Region-agnostic by design · All infra in AWS CloudFormation Two-account strategy: CodePipeline + ECR (Shared CI/CD) → Workload · Dev/prod by stack naming convention

Current deployment: ap-southeast-1 (Singapore). Every layer is configurable to any AWS region without architectural changes. Dashed line indicates the SQS-mediated asynchronous path from ECS to the IBKR Gateway.

3.

AWS SERVICE INVENTORY

Every AWS service in production use, its role, and design rationale.

COMPUTE

SERVICE	ROLE IN VEKTOR	DESIGN RATIONALE
ECS (Fargate)	All application microservices	Container-native deployment—each microservice is independently deployable and scalable. Fargate eliminates EC2 cluster management. **ECS Service Connect** handles inter-service communication via a local proxy that eliminates DNS lookup latency, providing consistent sub-100ms service-to-service calls. **Fargate Spot** is used for 80–100% of ECS workloads, delivering approximately 70% reduction in compute costs versus on-demand pricing.
EC2 t3.medium	Bastion host	Secure SSH access point for infrastructure administration. Isolated from the application layer.
EC2 t3.large	IBKR Gateway (SQS consumer)	Interactive Brokers requires a persistent gateway process—EC2 provides the stable, long-running process the IBKR API requires. Currently connected to IBKR simulation account.
SageMaker	Two roles: (1) JupyterLab research environment; (2) XGBoost model training, versioning, managed inference endpoint	Separating research from production inference is a governance decision. The notebook environment cannot directly affect production signals—model promotion requires an explicit deployment step.

DATA

SERVICE	ROLE IN VEKTOR	DESIGN RATIONALE
RDS PostgreSQL	Primary application DB + market data (equity prices, FX rates)	Single relational database—deliberate simplicity at prototype stage. ACID compliance important for allocation records forming the audit trail. **Schema separation:** three schemas—market_data, portfolio_data, customer_data—within a single RDS instance provide logical isolation today and a clear migration path to separate databases when scale requires it.
S3	Model artefacts + backup	SageMaker model artefacts stored in S3 enable model versioning and rollback. Also serves as backup target for RDS snapshots.

INTEGRATION & MESSAGING

SERVICE	ROLE IN VEKTOR	DESIGN RATIONALE
API Gateway	All ECS microservices exposed. Currently serves Admin Portal only.	Single entry point enforces authentication and logging consistently. **Rate limiting and WAF protection are on the near-term roadmap—not yet enforced.** External integrations (PMS, custodian) connect through the same gateway when added.
SQS	Market order queue between ECS and IBKR Gateway	Decouples order generation from execution. If IBKR is temporarily unavailable, orders queue rather than fail. Order record exists in SQS before execution: a pre-execution audit trail.
EventBridge	Scheduled triggers (price download, market orders)	Triggers Step Functions rather than Lambda directly—provides observable, retryable workflow orchestration with per-step logging.
Step Functions	Workflow orchestration for scheduled operations	Per-step logging, retry logic, visual workflow state. Each step is individually observable and retryable—contributes to the audit trail.
SNS + Lambda router	Notification fan-out architecture	Services publish events to SNS without knowledge of notification targets. A Lambda-based router delivers to Slack today. The architecture supports future multi-platform delivery (email, Teams, webhooks) without changes to publishing services — notification channel decisions are operational, not architectural.
Slack	Current notification delivery target	Receives operational notifications via SNS/Lambda router (orders, daily price download). CloudWatch alerting will supplement this as the platform moves toward production.

AUTH, SECURITY & FRONTEND

SERVICE	ROLE IN VEKTOR	DESIGN RATIONALE
Cognito	User + client authentication, Azure EntraID federation	Institutional firms using Microsoft 365 authenticate via existing SSO. Designed in from first prototype—not retrofitted when required.
KMS	Primary encryption at rest	AWS KMS for RDS and S3. All connections HTTPS/TLS. No unencrypted paths.
CloudWatch	Logging for all services	Aggregates logs from ECS, API Gateway, Step Functions. Alerting and dashboards on near-term roadmap.

SERVICE	ROLE IN VEKTOR	DESIGN RATIONALE
Amplify	React Admin Portal hosting + CI/CD	Connects to CI/CD pipeline. Handles frontend deployment automatically on merge to main.

4.

ARCHITECTURE — DATA FLOWS

How data moves through the platform—from market data ingestion through portfolio construction to order execution.

FLOW 1 — DAILY MARKET DATA PIPELINE (AUTOMATED)

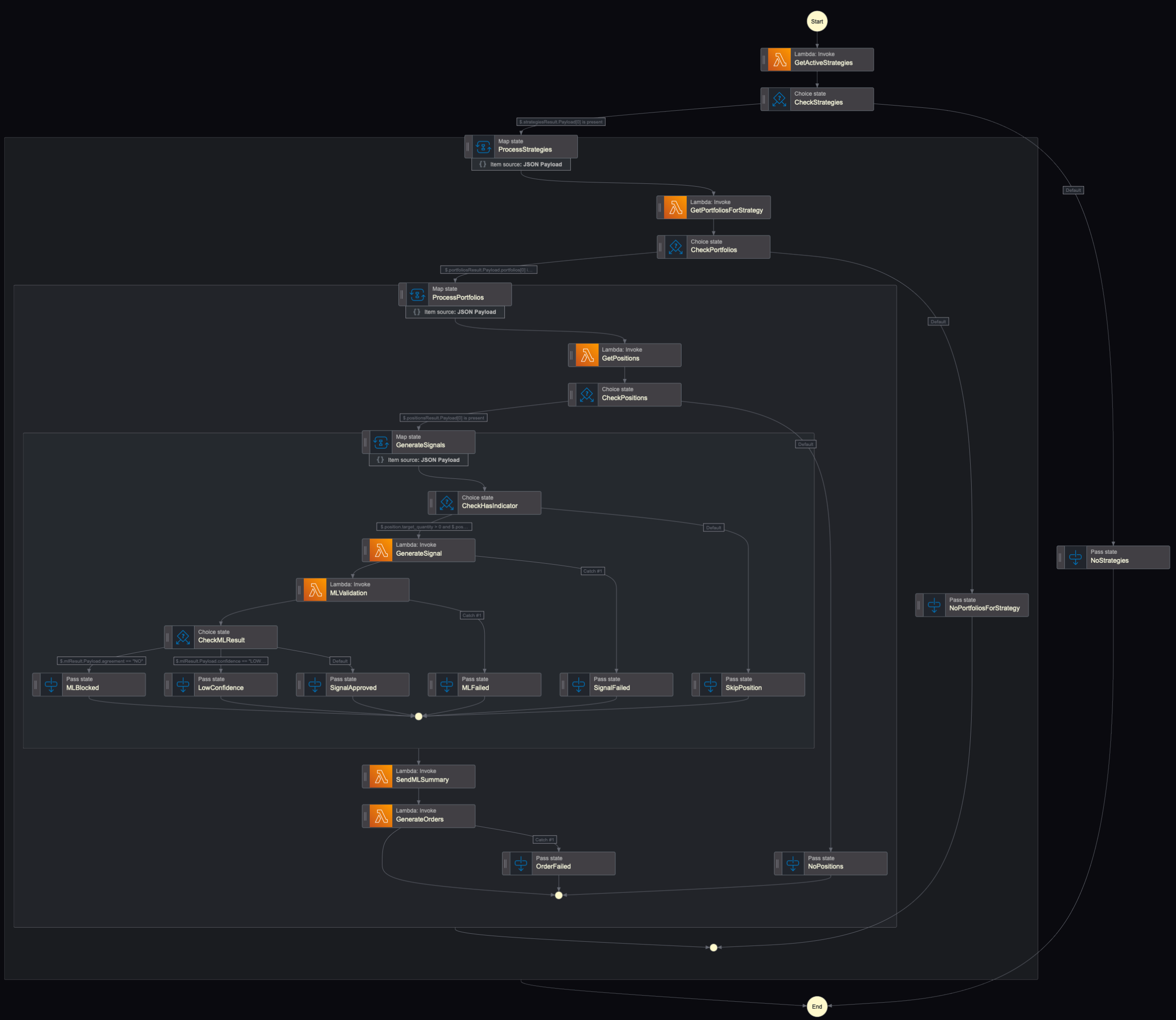
TIME	SERVICE	ACTION
08:00 SGT	EventBridge	Scheduled trigger fires—daily price download workflow
08:00 SGT	Step Functions	Workflow initiates—per-step logging begins
08:01 SGT	ECS data svc	Equity prices fetched for all configured instruments
08:02 SGT	ECS data svc	FX rates fetched for all configured currency pairs
08:03 SGT	RDS PostgreSQL	Price and FX records written—3 years daily history maintained
08:04 SGT	SNS → Lambda → Slack	Event published to SNS; Lambda router delivers to Slack: daily price download complete

FLOW 2 — RESEARCH TO STRATEGY TO FUNDED POSITIONS

STAGE	SERVICES	DETAIL
Research	SageMaker JupyterLab	Analyst: universe screening (500+ → target). **Portfolio optimisation:** 10,000 portfolio configurations generated to map the efficient frontier; Modern Portfolio Theory (PyPortfolioOpt) identifies the maximum Sharpe ratio portfolio from the mapped space. **Technical indicator backtesting and selection** across six indicator types (SMA, EMA, RSI, MACD, Bollinger Bands, Stochastic Oscillator)—system backtests each indicator per instrument and selects the best performer. XGBoost validation via SageMaker endpoint.
Strategy record	ECS → RDS	Outputs written to DB: instruments, weights, indicators, Sharpe ratios. Status: pending PM approval.

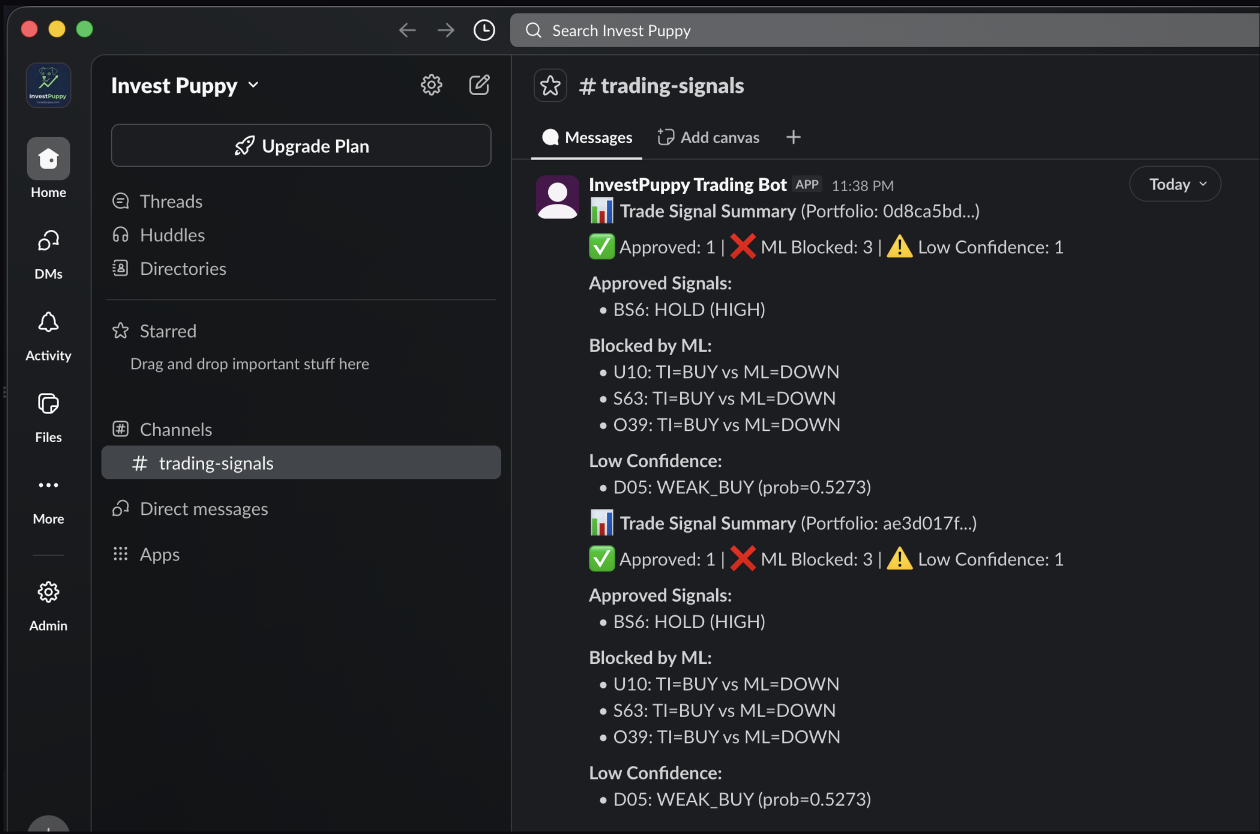
STAGE	SERVICES	DETAIL
Strategy approval	Admin Portal → API GW → ECS → RDS	PM reviews complete strategy. Approval writes audit record: PM identity + timestamp + strategy ID. **Maker/checker approval workflow with RBAC enforcement is planned for the next prototype iteration. Current implementation allows the PM to assign strategies directly.**
Client onboarding	Admin Portal → API GW → ECS → RDS	PM creates client profile, assigns approved strategy, sets mandate capital. Separate action from strategy approval.
Allocation	ECS → RDS → ECS → RDS	Allocation service fetches live prices, applies lot rounding, calculates position set. PM confirms. Audit record written.
Cash funding	Admin Portal → API GW → ECS → RDS	Operations records wire: currency, amount, reference. Cash account activated. Audit record written.
Order execution	ECS → SQS → EC2 → IBKR API	Trade instructions queued in SQS. IBKR Gateway polls and submits. Currently: simulation account. SNS publishes completion event; Lambda router delivers Slack notification.

PLATFORM EVIDENCE



AWS Step Functions order workflow — the complete pipeline from GetActiveStrategies through MLValidation to GenerateOrders, with explicit state handling for NoStrategies, NoPortfolios, MLBlocked, LowConfidence, SignalApproved, and SkipPosition outcomes.

PLATFORM EVIDENCE



InvestPuppy Trading Bot in the #trading-signals Slack channel — live signal summary showing Approved, ML Blocked, and Low Confidence signals per portfolio. The notification is generated automatically by the SNS→Lambda router after each execution cycle.

5.

THE DECISION LOG

Architectural decisions made at the design stage. Read in conjunction with the architecture diagram in Section 2.

DECISION MADE	ALTERNATIVE CONSIDERED	REASON
ECS for all microservices	Single EC2 monolith	Each service updates, scales, and restarts independently. A bug in the allocation service does not require redeploying the data service.
ECS Service Connect for inter-service comms	Standard DNS service discovery	Local proxy eliminates DNS lookup latency, providing consistent sub-100ms service-to-service calls without requiring a separate service mesh infrastructure component.
Fargate Spot for ECS compute	Fargate on-demand	80–100% Fargate Spot capacity delivers approximately 70% reduction in compute costs versus on-demand pricing. Compute cost is one component of total infrastructure cost; the saving is meaningful at scale without requiring architectural changes.
SQS between ECS and IBKR Gateway	Direct API call from ECS to IBKR	Direct calls create a hard dependency—if IBKR is unreachable, the flow fails. SQS decouples generation from execution. Order record exists in SQS before execution: a pre-execution audit trail.
EventBridge + Step Functions	Cron-triggered Lambda	Step Functions provides per-step logging, retry logic, visual workflow state. A Lambda cron produces one log entry; Step Functions produces a complete workflow audit trail.
SageMaker endpoint for XGBoost	Model embedded in ECS microservice	Managed endpoint allows model versioning and rollback independently of the application deployment. Model governance requires these concerns to be separated.
Cognito + Azure EntraID	Application-layer username/password	Institutional firms expect SSO. Designed in from first prototype—not added when an enterprise client required it.

DECISION MADE	ALTERNATIVE CONSIDERED	REASON
Two-account AWS strategy (Shared CI/CD + Workload)	Single AWS account	Separates deployment pipeline from application environment. CodePipeline in the shared account builds Docker images, pushes to ECR, and deploys cross-account to the workload account—this is the mechanism that makes the region-agnostic claim concrete. Current prototype uses a single workload account with environment separation by naming convention (dev/prod stacks). For production clients, separate workload accounts per environment or per client can be provisioned; the pipeline targets a different account ID with no architectural change required.
AWS CloudFormation for all infrastructure	Manual console configuration	All infrastructure defined as code. Every resource is version-controlled, reproducible, and auditable. No manual console configuration exists in the production path. Directly supports the "deploying to a new region is a pipeline configuration" claim—the CloudFormation stacks are the artefacts the pipeline deploys.
Singapore (ap-southeast-1) as primary region	US East or multi-region from day one	Data residency aligned with initial SGX market and MAS regulatory context. Architecture is region-agnostic—any region configurable without code change.
Single RDS with schema separation	Separate time-series DB for market data	Deliberate simplicity at prototype stage. Three schemas—market_data, portfolio_data, customer_data—provide logical isolation within a single RDS instance. Logical schema separation organises data and enforces query boundaries; it is not equivalent to physical database separation for compliance-grade data isolation. Physical separation into distinct database instances is the defined migration path for production-scale multi-client deployments. A deferral with a defined trigger condition, not an oversight.
API Gateway as single entry point	Direct ECS service calls	Authentication and logging enforced consistently today. Rate limiting and WAF protection on the near-term roadmap. External integrations connect through the same gateway—no additional exposure surface created.

6.

MODULARITY AND INTEGRATION CAPABILITY

How Vektor's architecture enables integration with existing institutional infrastructure—as a complement, not a replacement.

Each service is independently accessible via API Gateway. A client with an existing PMS does not need to replace it. Vektor operates as the quantitative construction and signal layer, passing outputs to existing execution and reconciliation infrastructure.

Allocation table output

Produces a complete position set—symbol, target quantity, target value, assigned indicator—as a database record retrievable via REST API. A PMS integration receives trade instructions in JSON. No file transfer.

Market data access

Equity price history and FX rates accessible via API Gateway. A client can call these endpoints directly, or substitute their own market data feed—the allocation engine accepts prices as input parameters.

Authentication federation

Cognito + Azure EntraID means Microsoft 365 users authenticate via existing SSO. No separate credential management required.

Audit log access

Every action logged as a timestamped record in RDS, accessible via API. A compliance team can query the complete decision chain for any position without accessing the Admin Portal.

> **Current status:** API Gateway serves the Admin Portal only—no external integration is yet live. The API layer is production-ready for external integration. Connecting to a specific PMS or custodian is a scoped integration project, typically two to four weeks for a standard REST integration.

7.

SECURITY AND DATA RESIDENCY

Encryption, access control, and data localisation.

Encryption at rest

AWS KMS for RDS and S3. All client data, market data, allocation records, and model artefacts are encrypted at rest.

Encryption in transit

All connections HTTPS/TLS. API Gateway enforces HTTPS for all endpoints. No unencrypted data paths.

Authentication and access control

Cognito with Azure EntraID federation. Infrastructure-level RBAC planned for next prototype—three-role model currently enforced at application layer.

Data residency

Current deployment: ap-southeast-1 (Singapore). Data does not leave the configured region. Clients with UK (eu-west-2), Hong Kong (ap-east-1), or US residency requirements deploy to the appropriate region—data residency guarantees follow. Full commitments available to Founding Mandate partners.

Network isolation

Services run within a VPC. Bastion host is the only SSH access point. API Gateway is the only public-facing application entry point. Full VPC architecture available on request.

8.

SCALABILITY

Scaling was a design requirement, not a roadmap item.

New regions without architecture changes

The two-account CI/CD strategy—CodePipeline building in the shared account and deploying cross-account via CloudFormation—means deploying Vektor to a new AWS region for a client in London, Dubai, or Hong Kong is a pipeline configuration. The shared account targets a different workload account and region; the CloudFormation stacks are the deployable artefacts.

Horizontal scaling via ECS

Each microservice scales horizontally. Adding mandates does not require architectural changes. Services scale independently. ECS Service Connect handles inter-service communication—sub-100ms calls regardless of instance count.

New markets without engine changes

Exchange, benchmark, currency, and lot sizes are RDS configuration parameters. Adding a new listed equity market is a configuration change, not a code deployment.

New clients in minutes

Client onboarding is a database operation. The Cognito user pool scales to accommodate new users without infrastructure changes.

SageMaker endpoint scaling

The XGBoost inference endpoint scales independently of the application layer during strategy creation when signal validation calls are most frequent.

Cost scaling discipline

Fargate Spot capacity means compute costs scale proportionally rather than linearly—the ~70% cost reduction versus on-demand is maintained as workloads grow.

9.

WHAT IS NOT YET IN SCOPE

Honest current state—and defined roadmap conditions.

Live trade execution

IBKR Gateway currently connected to simulation account. Full order flow operational in simulation mode. Live execution on near-term roadmap.

Infrastructure-level RBAC

Three-role model enforced at application layer. Infrastructure-level RBAC (IAM roles aligned to three-role model) planned for next prototype iteration.

Maker/checker approval workflow

Strategy assignment currently allows the PM to act directly. Maker/checker workflow with RBAC enforcement—requiring a separate checker to approve before a strategy is activated—is planned for the next prototype iteration.

API Gateway rate limiting and WAF

Authentication and logging are enforced at API Gateway today. Rate limiting and WAF protection are on the near-term roadmap.

Production observability

CloudWatch logging is in place. Alerting and dashboards not yet implemented. SNS/Lambda routing to Slack provides operational visibility for key events. Full CloudWatch alerting on near-term roadmap.

Production environment promotion

Current workload runs in the dev environment (stack naming convention). Two-account CI/CD architecture is in place and operational. The workload will be promoted to a separate production AWS account before the first Founding Mandate client is onboarded—a defined trigger condition, not an open-ended roadmap item.

External API integrations

All ECS microservices exposed via API Gateway and production-ready for external integration. No external PMS or custodian integration currently live. Integration with a specific external system is a scoped project, not an architectural change.

> These are sequencing decisions, not architectural gaps. The infrastructure designed today supports all of these capabilities. Implementing them is configuration and integration work, not architectural change.

10.

CONCLUSION

The architecture is not a collection of services—it is a set of decisions, made in a specific sequence, for specific reasons.

Singapore is the current deployment region—a starting point, not a constraint. Any AWS region can be configured as the deployment target for a client with different data residency or regulatory requirements. The two-account CI/CD architecture—CodePipeline, ECR, and CloudFormation stacks in a shared account deploying cross-account to the workload—makes this a pipeline configuration, not a rebuild.

The infrastructure choices documented here—ECS with Service Connect and Fargate Spot, CloudFormation-defined stacks, SNS-decoupled notifications, schema-separated RDS—reflect the same design principle applied consistently: build for production credibility from the first prototype, sequence implementations deliberately, and document what is live versus what is on the roadmap.

Full technical documentation is available to Founding Mandate partners as part of the due diligence process.

For compliance and audit trail architecture, see WP-06. For AI and ML governance, see WP-08. For the platform overview, see the Platform Brochure. All documents at investpuppy.com.